

Deep Learning for Solving Dynamic Models

Session 9 — Economics-informed neural networks: economic and financial applications

Simon Scheidegger

HEC, University of Lausanne

Based on Raissi, Perdikaris & Karniadakis (2019), *Journal of Computational Physics* 378, 686–707

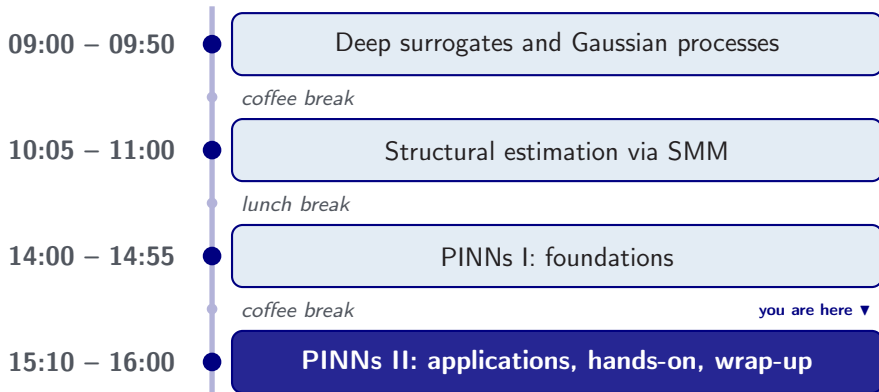
University of Padova

September 24, 2026

Further: Sirignano & Spiliopoulos (2018), Deep Galerkin Method for PDEs in high dimensions.

Al-Aradi, Correia, Naez, Hoshisashi & Schwab (2018), DGM PDE solver.

Day 2 at a Glance



Outline

1. From DEQNs to PINNs

Bridging discrete and continuous time; the PINN loss.

2. PINN Fundamentals

Automatic differentiation for PDEs, collocation, training loop.

3. Soft vs. Hard Boundary Conditions

Trial solutions, 2D Poisson, practical guidance.

4. Economic Applications: HJB Equation

Cake-eating problem, scaled trial solution + hard BCs, Adam $\rightarrow$ L-BFGS, analytical benchmark; DGM as optional high-D architecture.

5. Financial Applications: Black–Scholes

Option pricing via PINNs, comparison with analytical formula.

6. When to Use One, and What Goes Wrong

PINNs vs. grid solvers, inverse problems, failure modes, and which notebooks to run.

Roadmap of This Session (50 min)

Part II of two. Session 8 built the method: a PDE residual on collocation points, and a deliberate choice between soft and hard boundary conditions. Now we put it to work.

IV. HJB equations (14 min)

- ▶ Cake-eating: scaled trial solution, hard BCs, closed-form benchmark
- ▶ DGM as an optional high-dimensional architecture

V. Finance: Black–Scholes (6 min)

- ▶ Option pricing against the closed form; Greeks for free by autodiff

VI. When to use one (10 min)

- ▶ PINNs vs. grid solvers; the other half — inverse problems
- ▶ Failure modes, and the multi-component loss returning from Session 3

Hands-on (12 min) — build a PINN from scratch, at the keyboard.

Wrap-up (8 min) — what to use when, and where to go next.

Part IV

Economic Applications: The HJB Equation

The Hamilton–Jacobi–Bellman Equation

Generic continuous-time optimization:

$$\rho V(\mathbf{x}) = \max_c \{u(c) + V_{\mathbf{x}} \cdot f(\mathbf{x}, c)\}$$

- ▶ $V(\mathbf{x})$: **value function** (unknown)
- ▶ ρ : discount rate
- ▶ $u(c)$: flow utility
- ▶ $f(\mathbf{x}, c)$: law of motion for state $\mathbf{x}$
- ▶ $V_{\mathbf{x}} = \nabla V$: gradient of the value function

This is a PDE! First-order (or second-order with diffusion).

⇒ **Perfect fit for PINNs**: the network approximates V , autograd gives $V_{\mathbf{x}}$, and the HJB residual is the loss.

Cake-Eating Problem: Setup

A household with wealth $a(t)$ chooses consumption $c(t)$ to maximize lifetime utility:

$$\max_{\{c(t)\}} \int_0^{\infty} e^{-\rho t} \frac{c(t)^{1-\gamma}}{1-\gamma} dt \quad \text{s.t.} \quad \dot{a} = r a - c$$

HJB equation: $\rho V(a) = \max_c \left\{ \frac{c^{1-\gamma}}{1-\gamma} + V'(a)(r a - c) \right\}$

FOC: $c^{-\gamma} = V'(a) \Rightarrow c^* = (V'(a))^{-1/\gamma}$.

Analytical solution (with $\kappa = \frac{\rho - (1-\gamma)r}{\gamma} > 0$):

$$V^*(a) = \frac{\kappa^{-\gamma}}{1-\gamma} a^{1-\gamma}, \quad c^*(a) = \kappa a$$

Parameters: $\gamma = 2$, $\rho = 0.05$, $r = 0.03 \Rightarrow \kappa = 0.04$.

PINN for the HJB Equation: Hard-BC Trial Solution

Anchor + mask + scaled MLP. Let $x(a) = (a - a_{\min}) / (a_{\max} - a_{\min}) \in [0, 1]$ and S a value-scale constant:

$$\hat{V}(a) = V^*(a_{\min}) + x(a)[V^*(a_{\max}) - V^*(a_{\min})] + x(a)(1 - x(a)) S f_{\theta}(x(a)).$$

Endpoints are exact *by construction*, so the boundary loss is identically zero and the optimizer focuses on the HJB residual only.

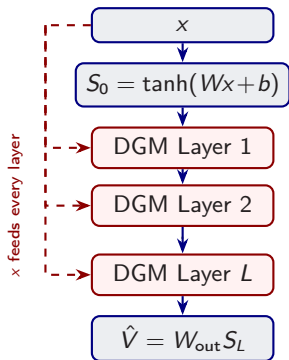
Consumption via FOC, $\hat{c}(a) = (\hat{V}'(a))^{-1/\gamma}$, with softplus safeguard on $\hat{V}'$.

Loss: only the interior HJB residual remains,

$$\ell = \frac{1}{N} \sum_i \mathcal{R}(a_i)^2, \quad \mathcal{R}(a) = \rho \hat{V} - \left[\frac{\hat{c}^{1-\gamma}}{1-\gamma} + \hat{V}'(ra - \hat{c}) \right].$$

Pedagogical point: a small MLP plus hard BCs beats a large gated net with a soft-BC penalty, because the boundary term no longer dominates the objective.

Optional Architecture for High-D PDEs: DGM



Deep Galerkin Method

(Sirignano & Spiliopoulos, 2018)

- Pays off only in **higher dimensions**, or where the PDE has sharp features.
- The input x enters **every** layer, not just the first — that is what resolves steep gradients.
- Each layer is **gated**, LSTM-style; the x -skips recall Highway Networks.

Not the default choice

For the 1D cake problem, the trial-solution MLP of the previous slide is simpler *and* more accurate.

Inside a DGM Layer

$S_\ell \in \mathbb{R}^U$ is the depth- ℓ hidden state, initialised as $S_0 = \tanh(Wx + b)$; $\odot$ is the Hadamard product; and Z, G, R are sigmoid gates valued in $[0, 1]$:

$$Z_\ell = \sigma(U^z x + W^z S_\ell + b^z)$$

$$G_\ell = \sigma(U^g x + W^g S_\ell + b^g)$$

$$R_\ell = \sigma(U^r x + W^r S_\ell + b^r)$$

$$H_\ell = \tanh(U^h x + W^h (S_\ell \odot R_\ell) + b^h)$$

$$S_{\ell+1} = (1 - G_\ell) \odot H_\ell + Z_\ell \odot S_\ell$$

Reading the gates

R_ℓ decides how much of the current state enters the candidate H_ℓ .

G_ℓ and Z_ℓ then blend that candidate against the state being carried forward.

Every one of them sees x directly.

Cost: four weight matrices per layer instead of one, so roughly $4\times$ the parameters of a plain MLP of the same width.

DGM Implementation (PyTorch)

```
class DGMNet(nn.Module):
    def __init__(self, dim, layers=4, units=64):
        super().__init__()
        self.input = nn.Linear(dim, units)
        self.z = nn.ModuleList(nn.Linear(dim, units) for _ in range(layers))
        self.wz = nn.ModuleList(nn.Linear(units, units) for _ in range(layers))
        # ... similar for g, r, h gates
        self.out = nn.Linear(units, 1)

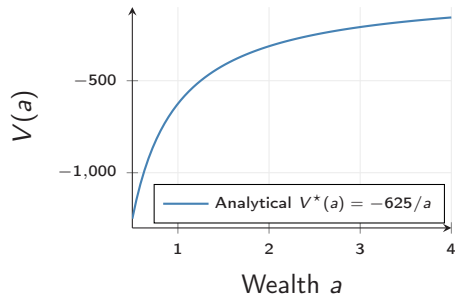
    def forward(self, x):
        s = torch.tanh(self.input(x))
        for l in range(len(self.z)):
            z = torch.sigmoid(self.z[l](x) + self.wz[l](s))
            g = torch.sigmoid(self.g[l](x) + self.wg[l](s))
            r = torch.sigmoid(self.r[l](x) + self.wr[l](s))
            h = torch.tanh(self.h[l](x) + self.wh[l](s * r))
            s = (1 - g) * h + z * s
        return self.out(s)
```

HJB Residual Implementation

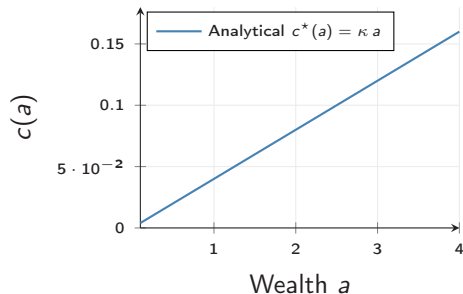
```
def pde_residual(model, a):  
    a.requires_grad_(True)  
    V = model(a) # value function  
    V_a = torch.autograd.grad( #  $V'(a)$  via autograd  
        V.sum(), a, create_graph=True)[0].squeeze(-1)  
  
    safe_Va = F.softplus(V_a) + 1e-6 # positivity guard  
    c = safe_Va.pow(-1.0 / gamma) # FOC: optimal c  
    u_c = c.pow(1-gamma) / (1-gamma) # flow utility  
  
    R = rho*V - (u_c + safe_Va*(r*a.squeeze(-1) - c))  
    return R # HJB residual
```

The softplus safeguard prevents $V'(a)^{-1/\gamma}$ from blowing up if $V'(a) \leq 0$ during early training.

Cake-Eating: the Closed-Form Benchmark



Value function $V(a)$ (closed form)



Consumption policy $c(a)$ (closed form)

These are the *exact* solutions, the target the PINN is scored against. Classroom run: HJB loss $\approx 3 \times 10^{-4}$, consumption relative L^2 error $\approx 6 \times 10^{-3}$. The notebook overlays the fitted $\hat{V}$ and $\hat{c}$.

Notebook: 04_Cake_Eating_HJB_PINN

Extensions: From Cake-Eating to Frontier Models

The PINN/DGM framework scales naturally:

1. **Income risk:** add diffusion $da = (ra - c + y)dt + \sigma dW \Rightarrow$ second-order HJB
2. **Aiyagari-type models:** HJB + Kolmogorov Forward Equation $\Rightarrow$ coupled PDE system
3. **General equilibrium:** prices clear markets $\Rightarrow$ hybrid DEQN + PINN
4. **High dimensions:** multiple assets, heterogeneous agents $\Rightarrow$ no grid needed!

See: Achdou et al. (2022), Fernandez-Villaverde et al. (2024), Gopalakrishna (2024).

Part V

Financial Applications: Black–Scholes

The Black–Scholes PDE

European call option (S : asset, K : strike, T : maturity):

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0$$

Terminal: $V(S, T) = \max(S - K, 0)$

Boundary conditions:

- ▶ $V(0, t) = 0$
- ▶ $V(S_{\max}, t) = S_{\max} - Ke^{-r(T-t)}$

PINN approach:

- ▶ Network $\mathcal{N}_\theta(S, t) \approx V(S, t)$
- ▶ Autograd: V_t, V_S, V_{SS}
- ▶ 4 loss terms:

ℓ_{PDE}	interior residual
ℓ_{BC_0}	$V(0, t) = 0$
ℓ_{term}	$V(S, T) = \text{payoff}$
$\ell_{\text{BC}_{S_{\max}}}$	far-field BC

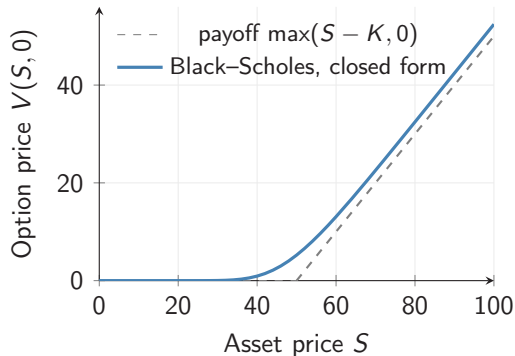
Black–Scholes PINN: Key Code

```
def pde_residual(model, S, t):  
    S.requires_grad_(True)  
    t.requires_grad_(True)  
    X = torch.cat([S, t], dim=1)  
    V = model(X)  
  
    V_t = torch.autograd.grad(V, t, ...)[0]  
    V_S = torch.autograd.grad(V, S, ...)[0]  
    V_SS = torch.autograd.grad(V_S, S, ...)[0]  
  
    # Black-Scholes PDE residual  
    res = V_t + 0.5*sigma**2*S**2*V_SS + r*S*V_S - r*V  
    return res
```

Loss:

$$\ell = \text{MSE}(\text{res}) + \text{MSE}(V|_{S=0}) + \text{MSE}(V|_{t=T} - \text{payoff}) + \text{MSE}(V|_{S_{\max}} - \text{target})$$

Black–Scholes: the Closed-Form Benchmark



Parameters: $K = 50$, $T = 1$, $r = 0.05$, $\sigma = 0.2$, $S_{\max} = 100$.

Network: 3 hidden layers, 64 units, tanh; 2,500 Adam epochs, then 300 L-BFGS iterations.

The gap between the curves is the **time value**: at the money the option is worth 5.23, not 0.

Against this benchmark the notebook reports max absolute error ≈ 0.13 , mean ≈ 0.04 , and plots the PINN overlay.

Notebook: 05_Black_Scholes_PINN

Part VI

When to Use One, and What Goes Wrong

Should You Use a PINN at All?

On a 1D problem, finite differences win. Outright.

Notebook 07 solves the same partial-equilibrium HJB both ways:

- ▶ **Upwind implicit FD**: 13 iterations on a 500-point grid, then the error is whatever the discretization gives you — and you can shrink it by refining.
- ▶ **PINN**: thousands of epochs to reach 10^{-3} – 10^{-2} , with no comparable knob.

Anyone selling you PINNs as a universal replacement for a grid solver is overselling.

Use a PINN when the state space is high-dimensional, the domain is irregular, there are many discrete states, or you need the solution *and* its derivatives as a differentiable object. Otherwise, use the grid.

Where the trade flips

A tensor grid needs n^d points. At $n = 50$:

$d = 1$ 50

$d = 2$ 2,500

$d = 4$ 6.3 million

$d = 6$ 1.6×10^{10}

Collocation cost is set by the *sample size you choose*, not by d .

The Other Half: Inverse Problems

So far the PDE was known and we solved it. Raissi et al. (2019) also treat the reverse.

Forward problem

Parameters μ given, solve for u .

$$\ell(\theta) = \|\mathcal{D}_\mu[\mathcal{N}_\theta]\|^2 + \text{BCs}$$

Inverse problem

Some data on u observed, solve for u and μ .

$$\ell(\theta, \mu) = \|\mathcal{D}_\mu[\mathcal{N}_\theta]\|^2 + \|\mathcal{N}_\theta - u^{\text{obs}}\|^2$$

- ▶ μ is just **another set of trainable parameters**: one extra tensor, same optimizer, same loop.
- ▶ In economics μ is a **structural parameter** — risk aversion, a discount rate, an adjustment cost — and the data term is a moment condition.
- ▶ This is estimation and solution in **one** optimization, rather than a solver nested inside an estimation loop.

Connects directly to the deep-surrogate approach to structural estimation from Day 1.

When PINNs Fail: the Multi-Component Loss, Again

The soft-BC loss is a weighted sum of residuals that live at different scales:

$$\ell_{\theta} = \underbrace{\lambda_{\text{pde}} \ell_{\text{pde}}}_{\text{interior}} + \underbrace{\lambda_{\text{bc}} \ell_{\text{bc}}}_{\text{boundary}} + \underbrace{\lambda_{\text{ic}} \ell_{\text{ic}}}_{\text{initial}}$$

This is exactly the Day 1 pathology.

- ▶ A residual carrying $S^2 \partial_{SS} V$ is not in the same units as one carrying V itself.
- ▶ The largest-scale component captures the gradient; the others are effectively ignored.
- ▶ Adaptive weights cannot close a gap of several orders of magnitude.

On Day 1 this bought $\approx 110\times$ over equal weighting, where the best adaptive scheme managed $2\times$ (02_DEQN.pdf, Part IV). See Wang, Teng & Perdikaris (2021).

What to do, in order

1. **Non-dimensionalize** first — rescale so every residual is $O(1)$.
2. Enforce **structurally** where you can: a hard-BC trial solution deletes ℓ_{bc} from the sum.
3. Only then reach for adaptive weights.

Hands-On: What to Run

All seven notebooks are PyTorch, CPU-only, and self-contained.

Run them on [nuvolos.cloud](#) — no local setup needed.

Session 8 — the method

- ▶ The simplest possible PINN: $y'' = -1$ with zero BCs 08_01
- ▶ Soft penalty vs. hard trial solution — the core choice 08_02
- ▶ 2D Poisson: hard BCs by transfinite interpolation 08_03

Session 9 — the applications

- ▶ The first economic application: the cake-eating HJB 09_01
- ▶ Black–Scholes, Greeks for free by autodiff 09_02
- ▶ PE HJB with discrete income, against an FD benchmark 09_03

Now: build one yourself 09_04

A fill-in-the-blank PINN for $u'' + u = 0$ on $[0, \pi]$, solution $\sin(x)$. We start it together now; solutions follow each task in the same file, so you can finish it afterwards.

Summary and Key Takeaways

Method

PINNs embed **differential equations** directly into the neural network loss function via **automatic differentiation**.

Boundary Conditions

Hard BCs (trial solutions) eliminate tuning and ensure exact satisfaction. Use **soft BCs** for complex domains.

Applications

HJB equations, Black–Scholes, and any continuous-time model. Scales to **high dimensions** without grids.

From DEQNs to PINNs:

- ▶ **Same philosophy:** economic/financial equations *are* the loss
- ▶ **New ingredient:** `torch.autograd.grad` for derivatives of $\mathcal{N}_\theta$
- ▶ **Broader scope:** any PDE, not just algebraic equilibrium conditions

Questions?

Simon Scheidegger

HEC, University of Lausanne

<https://sischei.github.io>

Materials: github.com/sischei/deep_learning_padova_0926 Deep Learning for
Continuous-Time Models

Companion Notebooks:

- 01 ODE PINN (zero BCs)
- 02 Soft vs. hard BCs
- 03 2D Poisson
- 04 Cake-eating HJB
- 05 Black-Scholes
- 06 Exercise: build a PINN
- 07 PE HJB: FD vs. PINN

Next up:

11:00–12:30 · Yucheng Yang